ShopVerse

Project Guide

A full-stack e-commerce platform with a wishlist, order tracking and an admin analytics dashboard

Live app
https://shopverse-web-lf22.onrender.com

Demo video
https://abhishek360007.github.io/ShopVerse/demo/

Source code
https://github.com/Abhishek360007/ShopVerse

Abhishek Raj · Java 25 · Spring Boot 4 · React 18 · PostgreSQL · Docker

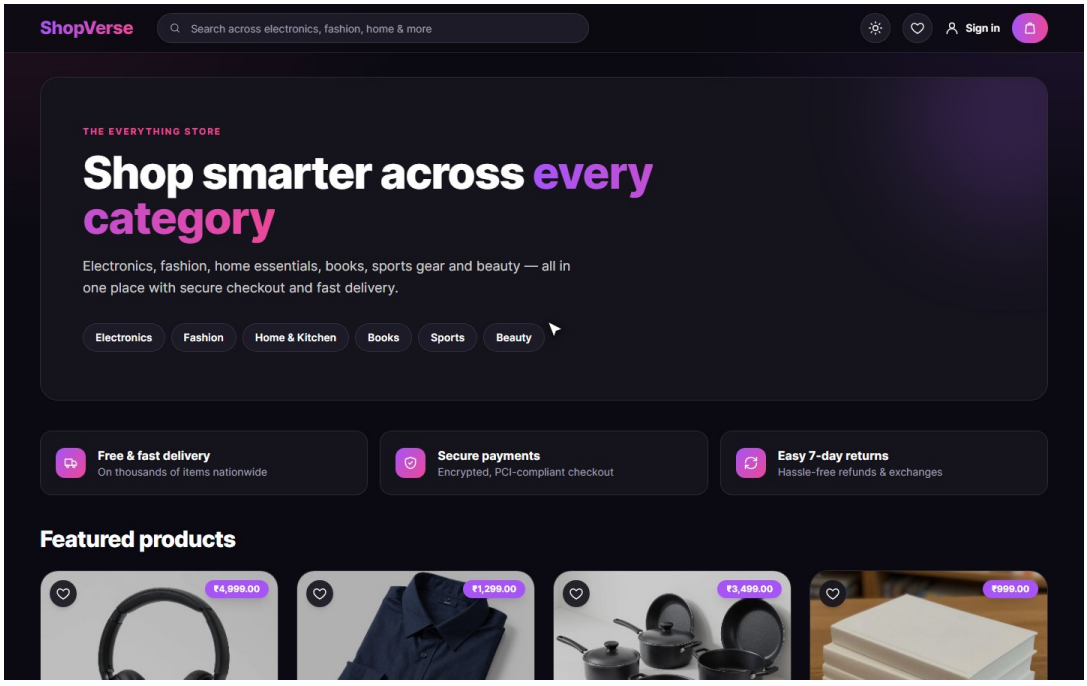
1. Overview

ShopVerse is an online store for electronics, fashion, home & kitchen, books, sports and beauty products. Customers browse and search the catalog, save products to a wishlist, check out with cash on delivery or a card, and follow each order through six delivery steps. Store admins get an analytics dashboard and manage orders, products and categories from an admin panel.

The app is split into a React single-page app and a Spring Boot REST API backed by PostgreSQL. It runs locally with one Docker Compose command and is deployed on Render with the database on Neon.

Highlights

- **Storefront**: 18 products in 6 categories with sub-type filters, full-text search, sizes and stock, light and dark themes.
- **Wishlist**: saved per account and synced across devices; items saved as a guest merge into the account at sign-in.
- **Checkout**: saved addresses, cash on delivery or Stripe card payments, with every order priced on the server.
- **Order tracking**: Placed → Confirmed → Packed → Shipped → Out for delivery → Delivered, each step timestamped.
- **Admin analytics**: revenue with month-over-month change, a monthly sales chart, best sellers, low-stock alerts and customer insights.
- **Security**: JWT authentication, Google sign-in, admin-only endpoints, CORS limited to the storefront's address.
- **Quality**: 81 backend and 32 frontend unit tests, run by GitHub Actions on every push.

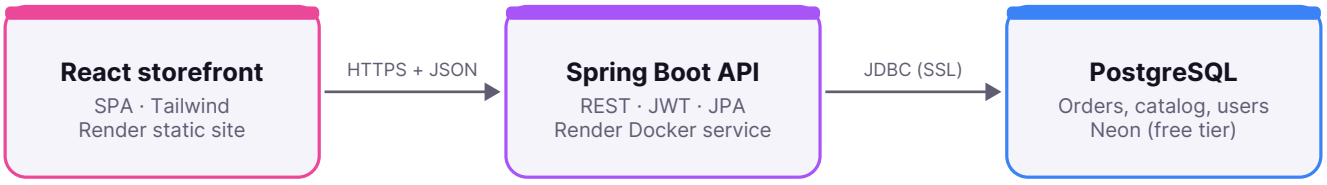


The storefront home page (dark theme).

2. Tech stack

Layer	Technology
Frontend	React 18, React Router 6, Redux Toolkit, Tailwind CSS, Axios
Admin panel	react-admin 5 with MUI, custom SVG sales chart
Backend	Java 25, Spring Boot 4, Spring Security, Spring Data JPA (Hibernate 7)
Auth	JWT (jjwt), Google OAuth2 sign-in
Database	PostgreSQL (Neon in production, Docker locally)
Payments	Stripe Payment Element (card), cash on delivery
Tooling	Maven, npm, Docker, Docker Compose, Nginx, GitHub Actions
Hosting	Render (API as a Docker web service, storefront as a static site), Neon (database)

3. Architecture



- The browser loads the static React app, which calls the API with a **JWT** in the Authorization header.
- Spring Security checks the token on every request; catalog reads are public, admin endpoints require the ADMIN role.
- A CORS filter runs before security so browser preflight requests are answered, and only the storefront's own address is allowed.
- On start-up the API runs small migrations and loads the starter catalog in numbered batches (see section 6).

Backend packages

Package	What it holds
auth	Login, registration, email verification, JWT filter, Google OAuth2, roles
controllers	REST endpoints: products, categories, wishlist, orders, admin orders, addresses, uploads
services	Business rules: order pricing, status rules, stock, analytics, wishlist
entities / repositories	JPA entities and Spring Data repositories
config	Security, CORS, start-up migrations, catalog seeder
dto / mapper	Request and response shapes, entity-to-DTO mapping

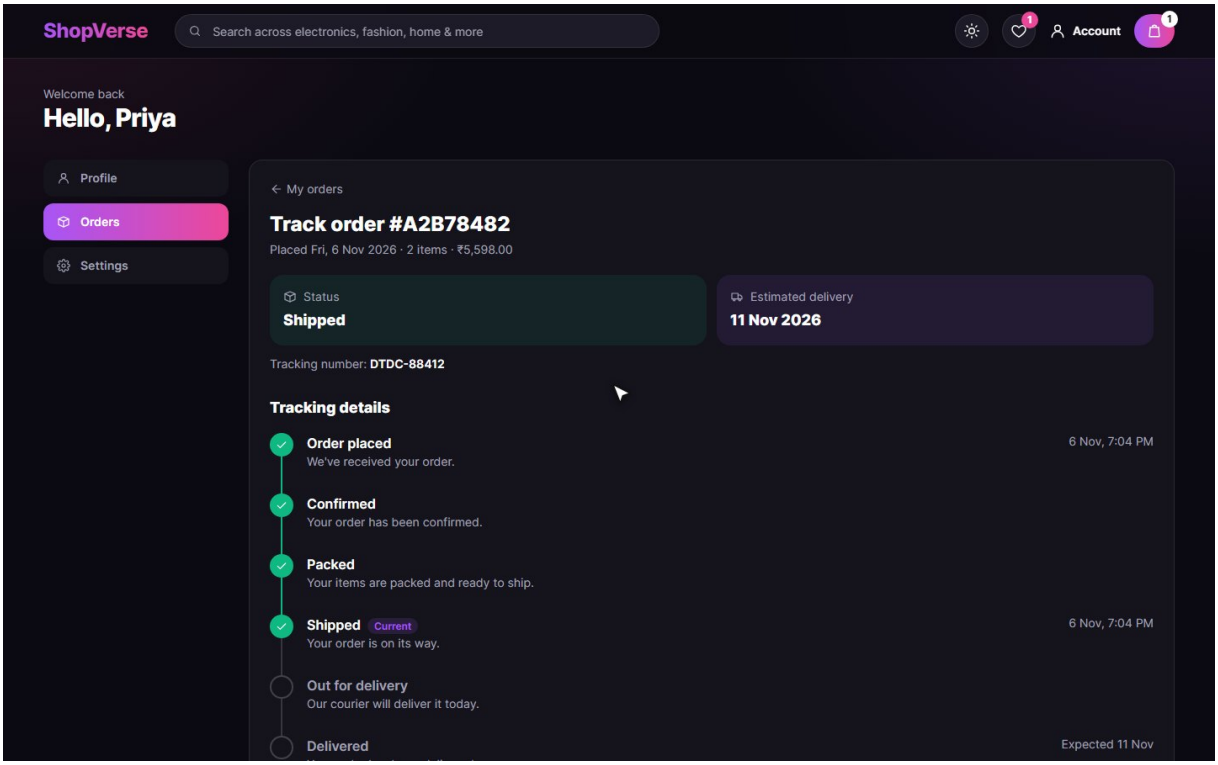
4. Features

Wishlist

A heart on every product card and product page saves the item. Signed-in customers' wishlists live in the wishlist_items table (one row per customer and product), so they appear on every device. Guests keep their wishlist in the browser; at sign-in the app sends those product IDs to /api/wishlist/merge, which adds new ones and skips duplicates or unknown products. The wishlist page moves items to the cart.

Checkout and server-side pricing

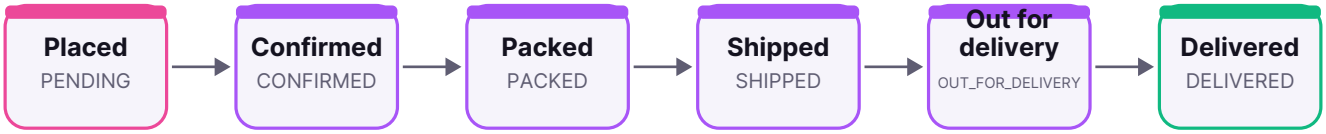
The client sends only product, option and quantity. **OrderPricing** looks up the catalog price, checks the option belongs to the product, validates the quantity (1-10) and stock, and calculates the total, so a tampered request can't change what is charged. Stock is reserved when the order is placed and returned if it is cancelled.



Order tracking: each delivery step with its time, the courier tracking number and estimated delivery.

Order tracking

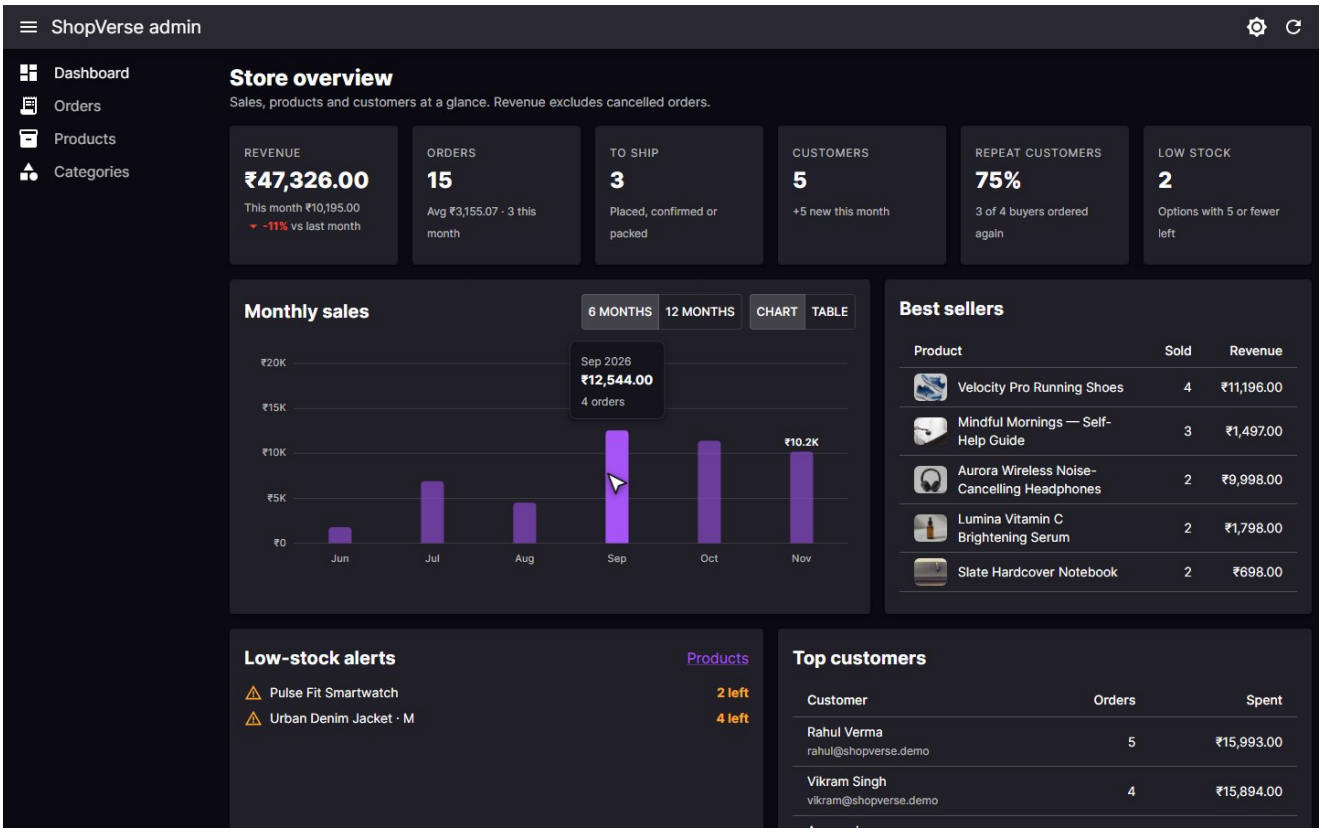
Orders move through six steps. Every change is stored in order_status_events with a timestamp and an optional note, which powers the customer's timeline and the admin's status history. **OrderStatusRules** only allows forward moves (admins may skip ahead) and cancellation before shipping; delivered and cancelled orders are final. Delivering a cash-on-delivery order marks it paid.



Cancelled (CANCELLED) is allowed from Placed, Confirmed or Packed.

Admin analytics dashboard

GET /api/orders/analytics returns everything the dashboard shows in one response: revenue and orders (excluding cancelled orders) with month-over-month change, monthly revenue for 6 or 12 months, the top five products by units sold, product options with five or fewer left, and customer figures (total, new this month, repeat buyers and top spenders). The monthly chart is a single-series SVG column chart with hover details and a table view.



Admin dashboard: KPIs, monthly sales, best sellers, low-stock alerts and customers.

Themes

All colours are CSS variables, so the light and dark themes switch instantly. The choice is saved in the browser, first visits follow the device setting, and a tiny script in index.html applies it before the first paint to avoid a flash.

5. Data model

Table	Purpose	Key relations
auth_user_details	Customers and admins	roles via auth_user_authority
categories / category_type	Six categories and their sub-types	category 1-n types
products	Name, price, brand, rating, slug	category, type; variants and images
product_variant	Size, colour, stock per option	product n-1
orders	Status, total, payment method, delivery date	customer, address; items, payment, events
order_items	Product, option, quantity, price paid	order n-1, product n-1
order_status_events	Tracking history with timestamps	order n-1
payment	Payment status and method	order 1-1
wishlist_items	Saved products (unique per customer)	customer n-1, product n-1
seed_history	Which catalog batches are loaded	-

6. Notable engineering decisions

- **Server-side pricing**: order totals are never trusted from the client.
- **Versioned catalog seeding**: products are added in numbered batches recorded in seed_history, so new products reach existing databases once, and products an admin deletes are not re-created on restart.
- **Start-up migrations**: when order statuses grew from five to seven, a migration replaced the database check constraint and renamed old IN_PROGRESS orders to CONFIRMED without data loss.
- **Readable API errors**: rule violations return 400 with a message (for example "Not enough stock"), and requests without a valid token get 401 instead of a login redirect.
- **Deploy-friendly config**: every secret comes from environment variables; the API listens on the platform's PORT and accepts host names with or without https://.

7. API reference

All responses are JSON. Send Authorization: Bearer <token> where a token is needed. A ready-made Postman collection (ShopVerse.postman_collection.json) covers every request below.

Method & path	Access	Description
`POST /api/auth/register`	Public	Create an account
`POST /api/auth/login`	Public	Returns a JWT
`GET /api/user/profile`	Signed in	Profile, addresses, roles
`GET /api/category`	Public	Categories with sub-types
`GET /api/products?categoryId&typeId&q&slug`	Public	List, filter, search or fetch by slug
`GET | POST | DELETE /api/wishlist[/{productId}]`	Signed in	Read, add or remove wishlist items
`POST /api/wishlist/merge`	Signed in	Merge a guest wishlist after sign-in
`POST /api/address`	Signed in	Save a delivery address
`POST /api/order`	Signed in	Place an order (priced on the server)
`GET /api/order/user`	Signed in	My orders
`GET /api/order/{id}`	Owner	Tracking view with status history
`POST /api/order/cancel/{id}`	Owner	Cancel before shipping
`GET /api/orders?status&page&size`	Admin	All orders, paged
`PUT /api/orders/{id}`	Admin	Change status, tracking number, note
`GET /api/orders/analytics`	Admin	Dashboard figures
`POST | PUT /api/products, /api/category`	Admin	Manage the catalog

8. Testing and CI

- **Backend (JUnit 5, Mockito)**: order pricing, status rules and stock, analytics calculations, role assignment, JWT handling, verification codes and URL handling.
- **Frontend (Jest)**: cart and wishlist reducers, price formatting, tracking timeline, theme, redirects, order requests.
- **GitHub Actions** runs both suites and a production build on every push to main.
- End-to-end checks were run against the full Docker stack, including the browser-recorded walkthrough in the demo video.

9. Running ShopVerse

Docker (quickest)

Step	Command
1. Configure	`cp backend/.env.example backend/.env (set JWT_KEY and ADMIN_EMAIL)`
2. Start	`docker compose up --build`
3. Open	`Store http://localhost:3000 · API http://localhost:8080 · Swagger /swagger-ui/index.html`

Without Docker

Part	Command
Backend (Java 25, PostgreSQL)	`cd backend && ./mvnw spring-boot:run`
Frontend (Node 18+)	`cd UI && npm install && npm start`
Tests	`cd backend && ./mvnw test · cd UI && npm test -- --watchAll=false`

Key environment variables

Variable	Purpose
`DB_HOST, DB_NAME, DB_USER, DB_PASSWORD, DB_SSLMODE`	Database connection (DB_SSLMODE=require for Neon)
`JWT_KEY`	Base64 secret for signing tokens
`ADMIN_EMAIL`	The account with this email becomes the store admin
`FRONTEND_URL, CORS_ALLOWED_ORIGINS`	The storefront's address for redirects and CORS
`EMAIL_VERIFICATION, MAIL_*`	Optional email verification
`STRIPE_SECRET, REACT_APP_STRIPE_KEY`	Optional card payments
`REACT_APP_API_BASE_URL`	Where the storefront finds the API

Deployment

The repository includes a Render Blueprint (render.yaml). Create a free Neon database, choose **New → Blueprint** on Render, select the repository and enter the Neon details and admin email. Pushing to main redeploys both services automatically.

10. Possible next steps

- Product reviews and ratings from verified buyers.
- Order confirmation and shipping emails.
- Coupons and discounts, and a paginated product catalog with sorting.
- Image uploads to cloud storage from the admin panel.